IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2021

INSTRUCTION ARCHITECTURES

Tuesday, 4 May 10:00 am

Time allowed: hours

There are TWO questions on this paper.

Answer questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : J. Wickerson, D.B. Thomas
Second Marker(s) :

© Imperial College London

INSTRUCTION ARCHITECTURES AND COMPILERS

1. a) How many bits are used to identify a register in MIPS R-type and I-type instructions, and why? [2]

   b) What types of token are “for”, “fox”, and “0x0” in the C grammar? [3]

   c) Give an example of a C statement which could map to a single CISC instruction, but maps to more than one MIPS instruction. [4]

   d) Give a MIPS assembly template for compiling C code of the form: [4]

```
while( C ) {
    X;
}
```

   e) Give an example of a situation where a C compiler can improve the temporal locality of a program. [4]

   f) Give an example of a C arithmetic expression which naïvely maps into two or more MIPS instructions, but could be implemented in one instruction. [4]

   g) Give a regex that is recognised by the Lex tool and matches your CID number, but without using any even decimal digits in the regex. [3]

   h) MIPS has a branch delay slot, but many modern RISC ISAs do not. State whether you think the delay slot is a good or a bad idea, and then briefly explain your reasoning. [4]

   i) Give two different ways of encoding a no-op using two different MIPS instructions which *do not* use the zero register. [4]

   j) A CPU with k pipeline stages is modified to have $k+1$ stages. Provide an upper bound on the improvement in IPC. [4]

   k) Write a context-free grammar that generates exactly the language accepted by the regular expression $\texttt{a}^*\texttt{b}^*\texttt{c}^*$. [4]

2. Imagine you have been asked to extend the MIPS ISA to support n-bit operations, where n is a binary power larger than 32. The register file will still contain 32 registers, and instructions are still 32 bits wide, but each register is now n bits. The purpose is to support large cryptographic operations, so it is anticipated that n might be quite large, for example 1024 or more.

a) Give an example of a MIPS R-type instruction which would not require any modification to its encoding in order to perform the same task in the new ISA. [4]

b) Give an example of a MIPS I-type instruction which cannot support the full set of operations over n-bit registers compared to 32-bit registers. [4]

c) If the ALU uses a two-level carry-select adder, what is the asymptotic critical path of the adder? [4]

d) The cost of a hardware multiplier has been deemed too expensive, so the designers have instead added two new instructions which allow multiplication to be perfomed in software:

MULX : Multiplies one n-bit register with the bottom 32-bits of another register, producing an n-bit result.

SHIFTX : Shifts a n bit register by $32i$ bits. The shift is left if $i < 0$ or logical right if $i > 0$.

i) Propose an instruction encoding for the MULX instruction using either the R-type or I-type format. [4]

ii) Give MIPS assembly code for full n-bit multiplication using the two new instructions. Assume the two inputs are in registers \$a0 and \$a1, the output should be placed in \$v0, and any other registers can be used as temporaries. [8]

iii) Often cryptographic operations require multiplications on numbers with just pn non-zero bits, where $p << 1$. Propose a new instruction which could take advantage of this property, and estimate the expected time complexity in terms of p and n. [6]

3. Consider the following grammar for programs.

$$
\begin{array}{rcl}
\textit{prog} & ::= & \textit{label}\texttt{:}\ \texttt{IFZERO}\ \textit{var}\ \texttt{GOTO}\ \textit{label}\texttt{;}\ \textit{prog} \\
& | & \textit{label}\texttt{:}\ \texttt{IFNONZERO}\ \textit{var}\ \texttt{GOTO}\ \textit{label}\texttt{;}\ \textit{prog} \\
& | & \textit{label}\texttt{:}\ \textit{var}\ \texttt{:=}\ \textit{expr}\texttt{;}\ \textit{prog} \\
& | & \varepsilon \\
\textit{expr} & ::= & \textit{var} \mid \textit{var}\ \texttt{/}\ \textit{var} \mid \textit{var}\ \texttt{/}\ \textit{literal} \\
\textit{literal} & ::= & \texttt{1} \mid \texttt{2} \mid \ldots \\
\textit{label} & ::= & \texttt{1} \mid \texttt{2} \mid \ldots \\
\textit{var} & ::= & \texttt{a} \mid \texttt{b} \mid \ldots \mid \texttt{z}
\end{array}
$$

All 26 variables are integer-valued and do not need initialising. The variable `a` is designated as the program's input and output. Labels and literals are positive integers and no two statements in a program have the same label. The `/` operator denotes integer division.

a) Devise a program in this grammar that can be improved by common subexpression elimination (CSE). Write down your program both before and after CSE has been applied. Explain why your program is better after transformation. [4]

b) Write down a program in this grammar that could be improved by CSE, but where a naïve analysis of available expressions would not lead to this optimisation opportunity being exploited. Explain the source of this imprecision. Suggest one way the analysis could be made more precise. [6]

c) Devise a program in this grammar that can be made *worse* by CSE. Write down your program both before and after CSE has been applied. Explain why your program is worse after transformation. [6]

d) Draw the control-flow graph (CFG) of the following program. [5]

```
1: a := 1;
2: IFZERO e GOTO 4;
3: b := a + b;
4: c := b + c;
5: IFNONZERO b GOTO 8;
6: d := d + d;
7: IFZERO d GOTO 3;
8: e := d;
```

e) Devise a loop-free CFG whose shape does not correspond to any program in the grammar. [2]

f) Devise a program in this grammar whose *register interference graph* has the following shape. Annotate your program with the set of live variables at the end of each line. [7]

